

The point of the story is that there is no need to “reinvent the wheel”; instead, you can simply add on to what you already know.

Inheritance in Object-Oriented Programming

Inheritance in OOP is the ability to create a class that builds on (*extends*) an existing class. When creating large programs, you will often use classes that provide very useful general capabilities. You’ll sometimes want to build a class that’s similar to a class that already exists, but does some things slightly differently. Inheritance allows you to do just that, creating a new class that includes all the methods and instance variables of an existing class, but adds new and different functionality.

Inheritance is an extremely powerful concept. When classes are set up correctly, using inheritance can *seem* simple. However, being able to design classes to use it in a clear manner is a skill that’s difficult to master. As an implementer, inheritance takes a great deal of practice to use properly and efficiently.

With inheritance, we talk about the relationship between two classes, typically referred to as the *base class* and *subclass*.

Base class

The class that is inherited from; it serves as a starting point for the subclass.

Subclass

The class that is doing the inheriting; it enhances the base class.

While these are the most common terms used to describe the two classes in Python, you may also hear them referred to in other ways, such as:

- *Superclass* and *subclass*